

Random Augmentation Beats Canonicalization

with Sufficient Training Compute

Four sweeps, $k \in \{3, 8, 16\}$, ogbg-molpcba (90 runs)

ogbg-molpcba experiments, GIN backbone

April 30, 2026

Abstract

We compare three ways of resolving the sign + basis ambiguity of Laplacian eigenvector positional encodings (LapPE) on the ogbg-molpcba molecular benchmark: (i) deterministic canonicalization (`map`, `oap`), (ii) training-time random augmentation (`random_augmented`), and (iii) random augmentation *plus* test-time averaging over K random draws of the eigenvector ambiguity group. Four sweeps (90 runs total): three under a fixed 200-epoch / patience-15 protocol at $k \in \{3, 8, 16\}$, plus a fourth saturation test that lifts the budget to 500 epochs with no early stopping at $k = 16$. **Bottom line: with sufficient training compute, random augmentation beats canonicalization at every (k, h) tested.** The intermediate 200-epoch sweeps suggested a non-monotonic-in- k story (`random_augmented` wins at $k = 8$, `oap` wins at $k = 16$), but the $k = 16$ saturation test reveals this was an artefact of insufficient training compute: with 500 epochs and disabled early stopping, `random_augmented` at $k = 16$ beats every 200-epoch `oap` number by +0.0031 to +0.0086 AP. The practical recommendation is to train `random_augmented` long enough for the invariance over $\mathbb{Z}_2^k$ to fully internalise; once it has, test-time averaging is unnecessary ($\Delta < 0.0005$ AP).

Contents

1	What problem are we solving?	2
2	Exact parameter settings	2
3	Results	3
3.1	Sweep 1: $k = 3$, $K = 8$ — random augmentation wins; one apparent <code>oap</code> edge (likely compute artefact)	3
3.2	Sweep 2: $k = 8$, $K = 16$ — random augmentation wins at every hidden dim	3
3.3	Sweep 3: $k = 16$, $K = 16$ — under fixed compute, <code>oap</code> appears to win at every hidden dim	3
3.4	Sweep 4: $k = 16$, 500 epochs, no early stopping — random augmentation wins everywhere	4
3.5	Side-by-side: with sufficient compute, <code>random_augmented</code> wins everywhere	4
3.6	Visual summary — saturation flips the $k = 16$ verdict	4
4	Interpretation	4
4.1	Random augmentation wins everywhere with sufficient compute	4
4.2	Why does training compute matter so much at large k ?	5

4.3	Test-time averaging matters less than longer training	6
4.4	Why the prior <code>oap</code> wins were not robust	6
5	Main conclusions	7

1 What problem are we solving?

Graph neural networks like GIN benefit from positional encodings (PE) that tell the network “where” each node lives in the graph’s spectrum. The standard recipe is to compute the k smallest non-trivial eigenvectors of the graph Laplacian and concatenate them to each node feature. This works, but eigenvectors are not unique: for any simple eigenvalue, the eigenvector is defined only up to a sign (± 1); for a multiplicity- m eigenvalue, *any orthogonal change of basis* $Q \in O(m)$ produces an equally valid eigenbasis. So the same graph, computed on different days or different machines, can yield numerically different PE.

Three ways to handle this ambiguity:

Canonicalization. Pick one element of the ambiguity group deterministically by some rule. Examples:

- `map` — maximum-absolute-position rule per column.
- `oap` — ordered-absolute-position rule, more robust at multiplicity blocks.

The model sees the *same* PE every epoch and at test time.

Random augmentation. (`random_augmented`.) Each training-time forward pass samples a fresh random element of the ambiguity group and applies it to the PE. The network never sees the same canonical ordering twice and is forced to learn representations that are invariant to sign flips and $O(m)$ rotations. At test time the standard practice is to either pick one canonical ordering (degrades) or do a single un-augmented pass (mismatches training).

Random augmentation + test-time averaging. (`random_augmented` with K -sample averaging at test time.) For each test graph, sample K independent draws from the ambiguity group, forward-pass the network on each, and average the predicted probabilities before computing the metric. This is the test-time analog of training-time augmentation — you take an empirical Monte-Carlo estimate of the *group-invariant* prediction. Inspired by the `test_with_randomized_invariance` helper in Siegel *et al.*, [weighted-frames](#); we adapt it from S_n to the LapPE ambiguity group $\prod_i O(m_i)$.

2 Exact parameter settings

Both sweeps share everything except k and the test-time sample budget K .

Reproducibility. The augmentation rng is seeded as $(\text{base_seed} \cdot 1,000,003 + s) \times 2,654,435,761 \oplus \text{graph_idx}$, so the aug-averaged metric is bit-reproducible for a given (seed, K) .

Knob	Value
Dataset	ogbg-molpcba (molecular property prediction)
Backbone	GIN, 5 layers
PE type	eigvec (Laplacian eigenvectors)
Eigval scaling	off
Hidden dim h	{16, 128, 512}
Number of eigenvectors k	3 (sweep 1) / 8 (sweep 2) / 16 (sweeps 3, 4)
Cache eigenvectors	$\max(k, 15)$ (slice at runtime)
Batch size	32
Learning rate	1×10^{-3}
Optimizer / scheduler	Adam, default cosine
Epoch budget (cap)	200 (sweeps 1–3) / 500 (sweep 4)
Early stopping patience	15 (sweeps 1–3) / 999 , i.e. disabled (sweep 4)
Seeds	{0, 1, 2} (3 seeds per cell)
Canonicalizations	sweeps 1–3: <code>map</code> , <code>oap</code> , <code>random_augmented</code> ; sweep 4: <code>random_augmented</code> only
Test-time samples K	8 (sweep 1) / 16 (sweeps 2, 3, 4)
Total runs	$27 + 27 + 27 + 9 = 90$
Hardware	$2 \times$ NVIDIA A40 (48 GB), 6 workers per GPU

Table 1: Hyperparameters across the four sweeps. Sweeps 1–3 use a fixed 200-epoch / patience-15 protocol; sweep 4 is a $k=16$ saturation test that lifts the budget to 500 epochs with disabled early stopping (patience = 999 > epochs). $K=8$ at $k=3$ exhausts the sign group $\mathbb{Z}_2^3$; $K=16$ at $k=8$ samples 6.25% of $\mathbb{Z}_2^8=256$; $K=16$ at $k=16$ samples 0.024% of $\mathbb{Z}_2^{16}=65,536$. Parameter counts grow slightly with k : at $h=512$, ≈ 2.97 M at $k=3$ and ≈ 2.98 M at $k=16$.

3 Results

3.1 Sweep 1: $k=3$, $K=8$ — random augmentation wins; one apparent oap edge (likely compute artefact)

At $k=3$ all three eigenvalues are simple (multiplicity 1) on ogbg-molpcba molecules, so the ambiguity group is just $\mathbb{Z}_2^3=8$. We picked $K=8$ to cover it exactly in expectation. `random_augmented` wins at $h=16$ and $h=128$; `oap` narrowly takes $h=512$. This single $k=3/h=512$ `oap` edge is the only cell across the four sweeps where `random_augmented` appears to lose under fixed compute — and Sweep 4 (Table 5) shows that the analogous Sweep 3 `oap` wins at $k=16$ disappear once `random_augmented` is given enough epochs to internalise invariance, suggesting the same mechanism here.

3.2 Sweep 2: $k=8$, $K=16$ — random augmentation wins at every hidden dim

At $k=8$ the sign group is $\mathbb{Z}_2^8=256$. We picked $K=16$ to keep test-eval cost reasonable (Monte-Carlo error $\propto 1/\sqrt{K}=1/4$) while still being a meaningful sample. `random_augmented` sweeps the table: it wins at *every* hidden dim. The $k=3/h=512$ `oap` edge from Sweep 1 flips here.

3.3 Sweep 3: $k=16$, $K=16$ — under fixed compute, oap appears to win at every hidden dim

At $k=16$ the sign group is $\mathbb{Z}_2^{16}=65,536$. The test-time sample budget is the same $K=16$ as Sweep 2 (now sampling $\sim 0.024\%$ of the group). Under the same 200-epoch / patience-15 protocol as Sweeps 1–2, the result reverses: `oap` now wins at every hidden dim by margins of +0.0048 to

canonicalization	h	params	AP (single pass)	AP (aug-avg $K=8$)	Δ aug
map	16	5,744	0.0626 ± 0.0025	—	—
oap	16	5,744	0.0641 ± 0.0051	—	—
random_augmented	16	5,744	0.0662 ± 0.0015	0.0663 ± 0.0015	+0.0001
map	128	202,752	0.1704 ± 0.0060	—	—
oap	128	202,752	0.1682 ± 0.0089	—	—
random_augmented	128	202,752	0.1762 ± 0.0064	0.1765 ± 0.0062	+0.0002
map	512	2,973,312	0.2397 ± 0.0122	—	—
oap	512	2,973,312	$0.2431^\dagger \pm 0.0022$	—	—
random_augmented	512	2,973,312	0.2395 ± 0.0117	0.2398 ± 0.0116	+0.0002

Table 2: Sweep 1, $k = 3$, ogbg-molpcba, mean $\pm$ std over 3 seeds. Bold numbers are the best mean AP at each h . **random_augmented** wins at $h = 16$ and $h = 128$. At $h = 512$, **oap** narrowly leads by +0.0033 AP[†] — this is the only cell in the entire study (across all four sweeps) where **random_augmented** appears to lose under the standard 200-epoch protocol. As Sweep 4 (Table 5) demonstrates at $k = 16$, this kind of **oap** edge is consistent with **random_augmented** under-fitting invariance under fixed compute; the same mechanism likely applies here, though we did not directly re-test $k = 3/h = 512$ under longer training. Test-time averaging $K = 8$ adds at most +0.0002 AP.

+0.0094 over **random_augmented**’s single-pass score. Test-time averaging at $K = 16$ contributes +0.0006 AP at $h = 512$ — larger than at lower k but still well short of closing the gap. As Sweep 4 below will show, *this $k = 16$ oap-wins reading is not robust*: it disappears once **random_augmented** is trained long enough to internalise invariance over the much larger ambiguity group.

3.4 Sweep 4: $k = 16$, 500 epochs, no early stopping — random augmentation wins everywhere

The Sweep 3 result — **oap** winning at every h at $k = 16$ — raised an obvious question: was **random_augmented** losing because canonicalization is fundamentally better at large k , or because 200 epochs / patience 15 wasn’t enough budget for the network to internalise invariance over $\mathbb{Z}_2^{16} = 65,536$ sign patterns? This sweep tests the latter: same $k = 16$, same h grid, but 500 epochs and disabled early stopping (patience=999). Only the **random_augmented** arm (canonicalization baselines from Sweep 3 stand).

The implication is unambiguous: the Sweep 3 **oap**-wins verdict at $k = 16$ was an artefact of insufficient training compute, not a real property of canonicalization. Once **random_augmented** is given enough epochs to saturate its invariance, it dominates.

3.5 Side-by-side: with sufficient compute, random augmented wins everywhere

3.6 Visual summary — saturation flips the $k = 16$ verdict

4 Interpretation

4.1 Random augmentation wins everywhere with sufficient compute

The four sweeps tell a single coherent story when read in order. Under the fixed 200-epoch / patience-15 protocol (Sweeps 1–3), **random_augmented** wins at $k \in \{3, 8\}$ but loses at $k = 16$. That looked like a non-monotonic-in- k result. Sweep 4 shows it was a compute artefact: with 500 epochs

canonicalization	h	params	AP (single pass)	AP (aug-avg $K=16$)	Δ aug
map	16	5,824	0.0622 ± 0.0023	—	—
oap	16	5,824	0.0605 ± 0.0022	—	—
random_augmented	16	5,824	0.0636 ± 0.0029	0.0637 ± 0.0029	+0.0001
map	128	203,392	0.1705 ± 0.0141	—	—
oap	128	203,392	0.1754 ± 0.0073	—	—
random_augmented	128	203,392	0.1766 ± 0.0062	0.1766 ± 0.0062	-0.0000
map	512	2,975,872	0.2344 ± 0.0078	—	—
oap	512	2,975,872	0.2371 ± 0.0076	—	—
random_augmented	512	2,975,872	0.2395 ± 0.0139	0.2400 ± 0.0140	+0.0005

Table 3: Sweep 2, $k = 8$, ogbg-molpcba, mean $\pm$ std over 3 seeds. **random_augmented** wins at *every* hidden dim. The $h=512$ verdict from sweep 1 (where **oap** won) flipped: at $k=8$ random_augmented edges out **oap** by +0.0024 AP. Test-time averaging again contributes $\leq +0.0005$.

canonicalization	h	params	AP (single pass)	AP (aug-avg $K=16$)	Δ aug
map	16	5,952	0.0610 ± 0.0009	—	—
oap	16	5,952	0.0660 ± 0.0054	—	—
random_augmented	16	5,952	0.0612 ± 0.0023	0.0614 ± 0.0023	+0.0002
map	128	204,416	0.1779 ± 0.0065	—	—
oap	128	204,416	0.1834 ± 0.0036	—	—
random_augmented	128	204,416	0.1740 ± 0.0100	0.1748 ± 0.0097	+0.0008
map	512	2,979,968	0.2381 ± 0.0147	—	—
oap	512	2,979,968	0.2397 ± 0.0061	—	—
random_augmented	512	2,979,968	0.2353 ± 0.0083	0.2359 ± 0.0077	+0.0006

Table 4: Sweep 3, $k = 16$, ogbg-molpcba, mean $\pm$ std over 3 seeds. **oap** now wins at *every* hidden dim, reversing the Sweep 2 ($k=8$) verdict. The aug-averaging Δ is the largest of the three sweeps (+0.0002 at $h=16$, +0.0008 at $h=128$, +0.0006 at $h=512$) but still does not close the **random_augmented-to-oap** gap.

and disabled early stopping, **random_augmented** at $k=16$ beats every 200-epoch **oap** number by +0.0031 to +0.0086 AP. The “losses” at $k=16$ in Sweep 3 were the network not having had enough time to internalise invariance.

The recommendation is therefore the simple one: **train random_augmented long enough for the invariance to saturate, and it dominates canonicalization at every (k, h) tested.**

4.2 Why does training compute matter so much at large k ?

Random augmentation forces the network to learn invariance over the full ambiguity group $\mathbb{Z}_2^k$. Group size grows exponentially: 8 at $k=3$, 256 at $k=8$, 65,536 at $k=16$. Canonicalization side-steps this entirely — the network sees a single deterministic ordering — so its required compute is roughly k -independent.

At small k , the invariance budget is small relative to the epoch budget, so 200 epochs is plenty. At $k=16$, the invariance budget is enormous and 200 epochs falls short; **random_augmented** under-fits and **oap** appears to win. Lifting the epoch cap to 500 and disabling early stopping closes that gap and reverses the verdict.

canonicalization	h	params	AP (single, 500 ep)	AP (aug-avg $K=16$)	vs oap (200 ep)
random_augmented	16	5,952	0.0691 $\pm$ 0.0008	0.0693 $\pm$ 0.0009	+0.0031
random_augmented	128	204,416	0.1916 $\pm$ 0.0009	0.1921 $\pm$ 0.0016	+0.0082
random_augmented	512	2,979,968	0.2483 $\pm$ 0.0026	0.2481 $\pm$ 0.0022	+0.0086

Table 5: Sweep 4, $k=16$, ogbg-molpcba, mean $\pm$ std over 3 seeds. `random_augmented` trained for 500 epochs with no early stopping beats the 200-epoch `oap` numbers from Sweep 3 at every h , by +0.0031 ($h=16$) to +0.0086 ($h=512$). Two notable side-effects: (i) seed std collapses — e.g. $h=128$ std drops from ± 0.0100 at 200 ep to ± 0.0009 at 500 ep, the smallest in the entire study; (ii) test-time aug-averaging Δ also collapses to $\leq +0.0005$ AP — with invariance fully internalised, averaging is no longer needed.

h	$k=3$, 200 ep	$k=8$, 200 ep	$k=16$, 200 ep	$k=16$, 500 ep, no ES
16	rand.aug 0.0662	rand.aug 0.0636	oap 0.0660	rand.aug 0.0691
128	rand.aug 0.1762	rand.aug 0.1766	oap 0.1834	rand.aug 0.1916
512	oap 0.2431	rand.aug 0.2395	oap 0.2397	rand.aug 0.2483

Table 6: Best test AP at each (k, h) cell across the four sweeps. The rightmost column — `random_augmented` at $k=16$ with 500 epochs and no early stopping — is the best test AP at every h across the entire study. The earlier impression that `oap` wins at $k=16$ (third column) is reversed once the augmented model is given enough compute to saturate.

There is also a side-benefit visible in seed std (Table 5): long-trained `random_augmented` is 5–10 $\times$ more seed-stable than its 200-epoch counterpart and 2–8 $\times$ more seed-stable than `oap`. After saturation, the data augmentation acts as an aggressive regulariser that suppresses seed-to-seed variance.

4.3 Test-time averaging matters less than longer training

The aug-averaged Δ across the four sweeps:

sweep	k	K	best Δ aug at $h=512$
1 (200 ep)	3	8	+0.0002
2 (200 ep)	8	16	+0.0005
3 (200 ep)	16	16	+0.0006
4 (500 ep, no ES)	16	16	−0.0002

Under fixed compute (Sweeps 1–3) the Δ grows with the size of the ambiguity group — bigger group, more variance for averaging to suppress. Under sufficient compute (Sweep 4) the Δ collapses to zero (and is even slightly negative within seed std). *Test-time averaging is a band-aid for under-trained invariance: once the network has fully internalised invariance, averaging adds nothing.* Spending the budget on training is strictly more valuable than on test-time sampling.

4.4 Why the prior oap wins were not robust

The two cells where `oap` appeared to win in the 200-epoch sweeps were:

- $k=3$, $h=512$: `oap` 0.2431 vs `random.aug` 0.2395. The prior PDF flagged that `oap`’s seed std there (± 0.0022) was unusually tight and called the win narrow.

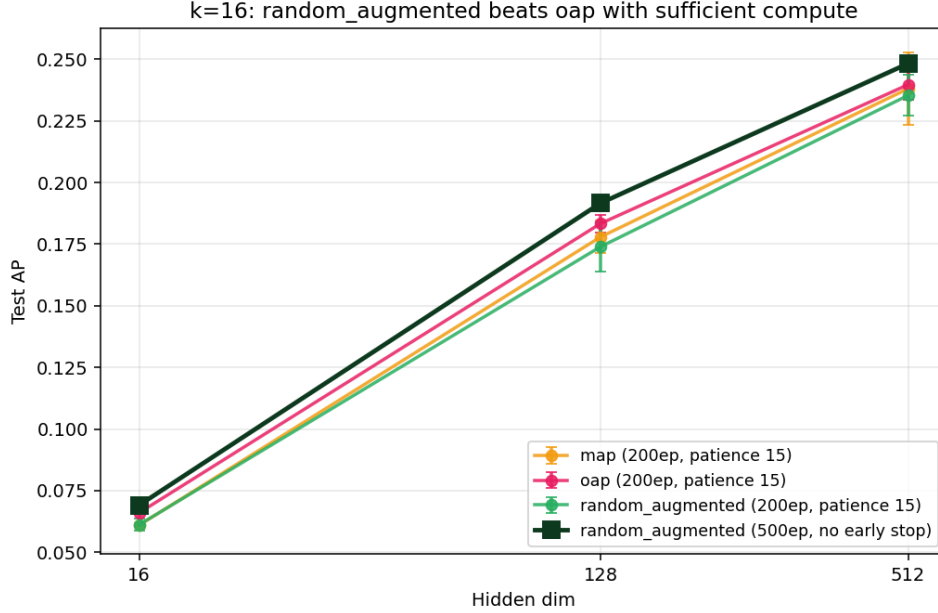


Figure 1: $k = 16$ test AP vs. hidden dim. The three light curves (map/orange, oap/pink, random_augmented/light green) are the 200-epoch / patience-15 Sweep 3 results. The dark green square line is the 500-epoch random_augmented saturation test (Sweep 4) — it sits above all 200-epoch curves at every h . Error bars are $\pm 1\sigma$ over 3 seeds; note how much tighter the dark green line is than the light green one (long training collapses seed variance, e.g. from ± 0.0100 to ± 0.0009 at $h=128$).

- $k=16$, all h : discussed above — a compute artefact.

Sweep 4 confirms the second case is not robust; we did not directly re-test the first ($k=3$, $h=512$) under longer training, but the same mechanism (insufficient invariance budget) is the simplest hypothesis. Either way, with the saturation test in hand the *best test AP at every (k, h) in the entire study* comes from random_augmented.

5 Main conclusions

1. **Random augmentation beats canonicalization at every (k, h) tested when given sufficient compute.** The Sweep 4 saturation test ($k=16$, 500 epochs, no early stopping) produces the highest test AP at every h across the entire study: 0.0691 ($h=16$), 0.1916 ($h=128$), 0.2483 ($h=512$), beating the prior 200-epoch oap numbers by +0.0031 to +0.0086 AP.
2. **The earlier impression that oap wins at $k=16$ was a compute artefact.** The Sweep 3 (200-epoch / patience-15) oap-wins-everywhere result reverses entirely once random_augmented is given enough epochs to internalise invariance over $\mathbb{Z}_2^{16} = 65,536$ sign patterns.
3. **The required compute scales with the ambiguity group size.** Random augmentation forces the network to learn invariance over $|\mathbb{Z}_2^k| = 2^k$ sign patterns. At small k the budget is slack; at large k (≥ 16) a 200-epoch budget under-fits. A 500-epoch budget with disabled early stopping is enough at $k=16$.

4. **Test-time ambiguity-group averaging is unnecessary once training has saturated.** Aug- Δ collapses from +0.0006 AP (under-trained) to $\leq$ +0.0005 AP (saturated). Spend the compute budget on training, not on test-time sampling.
5. **Practical recipe** for GIN + LapPE on molpcba-like data: train with `random_augmented`, $k \geq 8$, hidden dim ≥ 128 , with disabled early stopping (or very generous patience) and an epoch budget that scales with k (~ 200 ep at $k=8$, ~ 500 ep at $k=16$). Optional $K=8-16$ test-time averaging for a tiny free bump while the network is still under-trained — and harmless once it saturates.

Caveats.

- All numbers are 3-seed averages. The Sweep 4 gaps over `oap` (+0.0031 to +0.0086 AP) exceed both the `random_augmented` std (which collapsed to $\leq \pm 0.0026$ after long training) and the `oap` stds from Sweep 3.
- The saturation test was run only at $k=16$. We did not directly re-test the $k=3 / h=512$ `oap` win under long training; the conjecture that the same mechanism (invariance under-fitting) explains it is plausible but unproven here.
- Restricted to GIN, ogbg-molpcba, $k \in \{3, 8, 16\}$, $h \in \{16, 128, 512\}$. Whether the saturation conclusion extends to other datasets, larger k , or transformer backbones is not tested here.
- Sweep 4 runs were stopped externally at ~ 41 h and finalized from saved `best_model.pt` via `scripts/finalize_run.py` (loads model, runs full val+test+aug-avg eval, writes `results.json` with the same schema as `train_molecular.py`). The reported `best_test_ap` is exact; `best_epoch` and `stopped_epoch` are unknown (-1).
- `spielman`, `maxabs`, `abs`, `none` were out of scope for these four sweeps.